



# CashControl

## Documentation Administrateur / Développeur

*Guide technique — installation, architecture, base de données, maintenance*

---

|                               |                                                        |
|-------------------------------|--------------------------------------------------------|
| <b>Dépôt GitHub</b>           | github.com/noahPoivre/CashControl                      |
| <b>Branche</b>                | master                                                 |
| <b>Package Android</b>        | com.example.depenses_personnelles                      |
| <b>Langage</b>                | Kotlin (100 %)                                         |
| <b>UI</b>                     | XML Layouts + AppCompat (Views classiques)             |
| <b>Base de données</b>        | SQLite via SQLiteOpenHelper (DepensesDB.db, version 1) |
| <b>Bibliothèque graphique</b> | MPAndroidChart v3.1.0 (PhilJay) — PieChart             |
| <b>minSdk / targetSdk</b>     | 24 / 36                                                |
| <b>compileSdk</b>             | 36                                                     |
| <b>JVM cible</b>              | Java 11                                                |

# 1. Architecture du projet

## 1.1 Structure des fichiers sources

| Fichier / Dossier                   | Rôle                                                                                |
|-------------------------------------|-------------------------------------------------------------------------------------|
| <b>MainActivity2.kt</b>             | Activité principale — affiche le budget, oriente vers les 4 autres écrans           |
| <b>ajout.kt (AjoutActivity)</b>     | Écran d'ajout de budget — saisie d'un montant, appel insertAjout()                  |
| <b>depense.kt (DepenseActivity)</b> | Écran de saisie d'une dépense — montant + spinner catégorie, appel insertDepense()  |
| <b>log.kt (LogActivity)</b>         | Écran historique — RecyclerView alimenté par getAllLogs()                           |
| <b>statistique.kt</b>               | Écran statistiques — PieChart MPAndroidChart alimenté par getDepensesParCategorie() |
| <b>DatabaseHelper.kt</b>            | Couche données — SQLiteOpenHelper, toutes les requêtes SQL                          |
| <b>LogAdapter.kt + LogItem</b>      | Adaptateur RecyclerView + data class pour l'historique                              |
| <b>ui/theme/</b>                    | Fichiers Compose (Color, Theme, Type) — inutilisés dans les écrans actuels          |

## 1.2 Layouts XML

| Layout                          | Activité associée                                                      |
|---------------------------------|------------------------------------------------------------------------|
| <b>activity_main.xml</b>        | MainActivity2 — budget card + 4 boutons                                |
| <b>activity_ajout.xml</b>       | AjoutActivity — titre, EditText montant, bouton Ajouter, bouton Retour |
| <b>activity_depense.xml</b>     | DepenseActivity — titre, EditText montant, Spinner catégorie, boutons  |
| <b>activity_log.xml</b>         | LogActivity — titre, CardView contenant un RecyclerView, bouton Retour |
| <b>activity_statistique.xml</b> | statistique — titre, CardView contenant le PieChart, bouton Retour     |
| <b>item_log.xml</b>             | Item de liste pour le RecyclerView (type, catégorie, montant)          |

## 1.3 Flux de navigation

MainActivity2 est le point d'entrée (LAUNCHER). Elle lance les 4 autres activités via `startActivity(Intent)`. Le retour se fait toujours par `finish()` (bouton Retour physique ou bouton de l'écran), ce qui déclenche `onResume()` dans MainActivity2 et actualise le budget affiché.

## 2. Base de données (SQLite)

### 2.1 Schéma

Le fichier DatabaseHelper.kt crée la base DepensesDB.db (version 1) avec 3 tables :

| Table            | Colonnes principales                                                    |
|------------------|-------------------------------------------------------------------------|
| <b>Categorie</b> | idCat INTEGER PK AUTOINCREMENT, nom TEXT NOT NULL                       |
| <b>Depense</b>   | idDep PK, montant REAL, date TEXT, description TEXT, idCat FK→Categorie |
| <b>Ajout</b>     | idAjout PK, montant REAL, date TEXT, idCat FK→Categorie (nullable)      |

À la création (onCreate), 4 catégories par défaut sont insérées : Alimentation, Transport, Loisirs, Autre.

La date est stockée avec datetime('now') — format SQLite UTC.

### 2.2 Méthodes disponibles dans DatabaseHelper

| Méthode                                             | Description                                                                                     |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>insertAjout(montant, idCat?)</b>                 | Insère un ajout de budget. idCat est nullable (les ajouts n'ont généralement pas de catégorie). |
| <b>insertDepense(montant, description?, idCat?)</b> | Insère une dépense avec sa catégorie.                                                           |
| <b>getAllCategories()</b>                           | Retourne List<Pair<Int,String>> — liste de (idCat, nom) pour alimenter le Spinner.              |
| <b>getAllLogs()</b>                                 | Retourne List<LogItem> — tous les ajouts puis toutes les dépenses avec leur catégorie.          |
| <b>getBudgetTotal()</b>                             | Retourne Double — SUM(Ajout.montant) – SUM(Depense.montant).                                    |
| <b>getDepensesParCategorie()</b>                    | Retourne Map<String,Double> — somme des dépenses groupées par catégorie (pour le PieChart).     |

### 2.3 Migration de base de données

La méthode onUpgrade() est destructive : elle supprime les 3 tables et les recrée (DROP + onCreate). Si vous incrémentez DATABASE\_VERSION, toutes les données utilisateur seront perdues. Pour une migration non destructive, remplacez onUpgrade() par des ALTER TABLE ciblés.

### 2.4 Accès à la base en développement

Via Android Studio — App Inspection > Database Inspector (appareil connecté en débogage) : permet de visualiser et modifier les tables en temps réel.

Via ADB :

```
adb shell run-as com.example.depenses_personnelles \  
    cp /data/data/com.example.depenses_personnelles/databases/DepensesDB.db \  
    /sdcard/  
adb pull /sdcard/DepensesDB.db .
```

## 3. Environnement de développement

### 3.1 Prérequis

| Outil                       | Version requise                                          |
|-----------------------------|----------------------------------------------------------|
| Android Studio              | Meerkat (2024.3.2) ou supérieur recommandé               |
| JDK                         | Java 11 (configuré dans compileOptions et kotlinOptions) |
| Android SDK                 | API 24 minimum, API 36 compileSdk                        |
| Gradle                      | Wrapper inclus — géré automatiquement (gradlew)          |
| AGP (Android Gradle Plugin) | 9.1.0 (défini dans libs.versions.toml)                   |
| Kotlin                      | 2.2.10                                                   |

### 3.2 Cloner et ouvrir

```
git clone https://github.com/noahPoivre/CashControl.git
```

Puis dans Android Studio : File > Open > sélectionner le dossier CashControl-master.  
Attendez la synchronisation Gradle.

### 3.3 Compiler et lancer

| Commande                                    | Description                                            |
|---------------------------------------------|--------------------------------------------------------|
| <code>./gradlew assembleDebug</code>        | Compile l'APK de debug                                 |
| <code>./gradlew installDebug</code>         | Installe le debug sur l'appareil USB connecté          |
| <code>./gradlew assembleRelease</code>      | Compile l'APK de release (non signé sans keystore)     |
| <code>./gradlew bundleRelease</code>        | Génère un Android App Bundle (.aab) pour le Play Store |
| <code>./gradlew test</code>                 | Lance les tests unitaires (ExampleUnitTest)            |
| <code>./gradlew connectedAndroidTest</code> | Lance les tests instrumentés (ExampleInstrumentedTest) |
| <code>./gradlew lint</code>                 | Analyse statique du code                               |
| <code>./gradlew clean</code>                | Nettoie les artefacts de build                         |
| <code>./gradlew dependencies</code>         | Liste toutes les dépendances résolues                  |

L'APK de debug est produit dans :

```
app/build/outputs/apk/debug/app-debug.apk
```

## 4. Dépendances

Les versions sont centralisées dans `gradle/libs.versions.toml`.

| Bibliothèque                                            | Version                                                       |
|---------------------------------------------------------|---------------------------------------------------------------|
| <code>androidx.core:core-ktx</code>                     | 1.17.0                                                        |
| <code>androidx.appcompat:appcompat</code>               | 1.7.1                                                         |
| <code>com.google.android.material:material</code>       | 1.13.0                                                        |
| <code>androidx.constraintlayout:constraintlayout</code> | 2.2.1                                                         |
| <code>androidx.cardview:cardview</code>                 | 1.0.0                                                         |
| <code>androidx.lifecycle:lifecycle-runtime-ktx</code>   | 2.9.4                                                         |
| <code>androidx.activity:activity-compose</code>         | 1.12.0                                                        |
| <b>Jetpack Compose BOM</b>                              | 2024.09.00 (présent mais non utilisé dans les écrans actuels) |
| <code>com.github.PhiJay:MPAndroidChart</code>           | v3.1.0 — graphique PieChart                                   |

 *Jetpack Compose est configuré (plugin + dépendances BOM) mais les écrans utilisent des layouts XML classiques (AppCompat + ConstraintLayout). La migration vers Compose est possible progressivement.*

## 5. Points d'amélioration identifiés

| Problème / Limitation                                                                    | Recommandation                                                                                                                                       |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deux déclarations de <code>rootProject.name</code> dans <code>settings.gradle.kts</code> | Supprimer la première ligne ( <code>depensespersonnelles</code> ), conserver <code>depenses-personnelles</code>                                      |
| <code>onUpgrade()</code> destructif — perte de données à chaque migration                | Remplacer par des <code>ALTER TABLE</code> ciblés selon <code>oldVersion</code>                                                                      |
| Injection SQL possible dans <code>onCreate()</code> (INSERT avec concaténation)          | Utiliser des <code>PreparedStatement</code> ou <code>SQLiteDatabase.insert()</code> avec <code>ContentValues</code>                                  |
| <code>isMinifyEnabled = false</code> en release                                          | Activer R8 + règles ProGuard pour réduire la taille et obfusquer le code                                                                             |
| Historique sans tri par date                                                             | Ajouter <code>ORDER BY date DESC</code> dans <code>getAllLogs()</code> et stocker la date sur chaque ligne d'historique                              |
| Pas de suppression de transaction possible                                               | Ajouter un <code>swipe-to-delete</code> sur le <code>RecyclerView</code> avec une méthode <code>deleteById()</code> dans <code>DatabaseHelper</code> |
| Catégories fixes non éditables                                                           | Ajouter un écran de gestion des catégories (ajout / suppression)                                                                                     |
| Pas de filtre par période dans l'historique                                              | Ajouter un filtre date dans <code>getAllLogs()</code> et <code>getDepensesParCategorie()</code>                                                      |
| Fichiers <code>Compose</code> inutilisés ( <code>ui/theme/</code> )                      | Supprimer si <code>Compose</code> n'est pas utilisé, ou migrer progressivement les écrans vers <code>Compose</code>                                  |

## 6. Sécurité et confidentialité

| Point                                   | État actuel                                                                    |
|-----------------------------------------|--------------------------------------------------------------------------------|
| <b>Transmission réseau</b>              | Aucune — app 100 % offline, pas de permission INTERNET dans le Manifest        |
| <b>Permissions Android déclarées</b>    | Aucune permission spéciale (backup uniquement)                                 |
| <b>Stockage des données</b>             | Base SQLite dans le répertoire privé de l'app — inaccessible sans root         |
| <b>Sauvegarde Android (allowBackup)</b> | Activé dans le Manifest — les données peuvent être sauvegardées via ADB backup |

Le projet ne collecte, ne stocke, ni ne transmet aucune donnée personnelle en dehors de l'appareil de l'utilisateur.